Science Robotics
AAAS

Supplementary Materials for

Swarm of micro flying robots in the wild

Xin Zhou *et al.*

Corresponding author: Fei Gao, fgaoaa@zju.edu.cn; Chao Xu, cxu@zju.edu.cn

Sci. Robot. **7**, eabm5954 (2022)
DOI: 10.1126/scirobotics.abm5954

The PDF file includes:

Sections S1 to S10
Figs. S1 to S17
Tables S1 to S13

Other Supplementary Material for this manuscript includes the following:

Software

Section S1. Extra Comparison of Reciprocal Collision Avoidance

Table S1. Extra performance comparison. Comparison is performed under a position swap configuration in an 8×8 m empty space containing eight agents with radius of 0.25 m. Maximum velocity and acceleration are set to 1.7 m/s and 6 m/s^2, respectively. **Min. Clear.** denotes the minimum distance from collision. **Smoothness** is computed from the time integral of squared jerk $\int ||j||_2^2 dt$. Units of time and distance are second and meter, respectively.

Online/ Offline	Method	Comp. Time	Traj. Time	Traj. Length	Min. Clear.	Safety	Smoothness
Offline	SCP, h_scp=0.3s	1.46	7.65	**9.64**	-0.37	No	7.01
	SCP, h_scp=0.17s	7.72	**7.40**	9.70	-0.26	No	17.5
	RBP,no batches	**0.320**	15.4	11.3	0.01	Yes	0.608
	RBP,batch_size=4	0.413	15.4	11.5	0.03	Yes	**0.604**
Online	Mader	0.0239	**7.50**	12.2	0.19	Yes	125.1
	EGO, horizon=7.5m	0.000554	8.12	10.1	0.17	Yes	72.0
	EGO, horizon=10m	0.000844	8.17	10.3	0.31	Yes	94.1
	Proposed, $\kappa = 5$	**0.000465**	7.57	**9.70**	0.09	Yes	30.4
	Proposed, $\kappa = 8$	0.000557	7.58	9.71	0.11	Yes	**28.2**

Except for **EGO** (*33*) and **MADER** (*37*), we compare the proposed planner against two other centralized offline swarm planners, i.e., **SCP** (*49*) and **RBP** (*50*). Results are given in Table S1. A bold term indicates a better value in the corresponding category (Online/Offline), while an underlined term indicates the best value among all planners. As given in Table S1, **SCP** achieves a shorter flight time at the sacrifice of trajectory safety. **RBP** achieves smoother flight at the cost of a relatively long flight time. Both fail to balance various aspects of trajectory quality. Compared with offline methods, the three online planners show more balance while requiring less computation by several orders of magnitude. Among them, **MADER** is more conservative because its safety is ensured through minimum volume enclosing simplices. All the data are recorded from 10 random runs.

Section S2. Scalability Evaluation

Time and Space Complexity

In real-world usage, larger swarms always mean a more powerful team for broader coverage in exploration, a grater possible load for transportation, and higher robustness against robot failures, among other advantages. However, traditional optimization-based trajectory planning suffers from a high computing complexity related to the number of robots, as stated by Park et al. (*50*) the time complexity of which is $O(n^3)$. Therefore, Park et al. have to divide the entire swarm into groups and then plan the trajectory sequentially, which only reliefs the complexity to a limited extent.

Owing to the decentralized architecture and appropriate problem formulation, our time complexity is significantly reduced to approximately linear to the drone number from statistics. Since the baseline of computing time is short, the swarm size can increase to hundreds of robots before breaking the real-time bottom line. Meanwhile, the space complexity is exactly linear to the drone number, as one drone only stores the latest trajectory from every other one. Indeed, this space usage can be ignored because one trajectory is typically below 1 kB in size, and there are several gigabytes in the system memory.

The complexity can be further reduced (*20*) if each drone only considers nearby units, since we perform continuous planning only within a limited distance. In our work, we ignore drones more than twice the planning distance (see Table S6) away from each other. The resulting complexity is below $O(n)$, and is determined by the density of drone distribution as in Fig. S1. Such complexity allows the swarm to be expanded to a large size before breaking the real-time threshold.

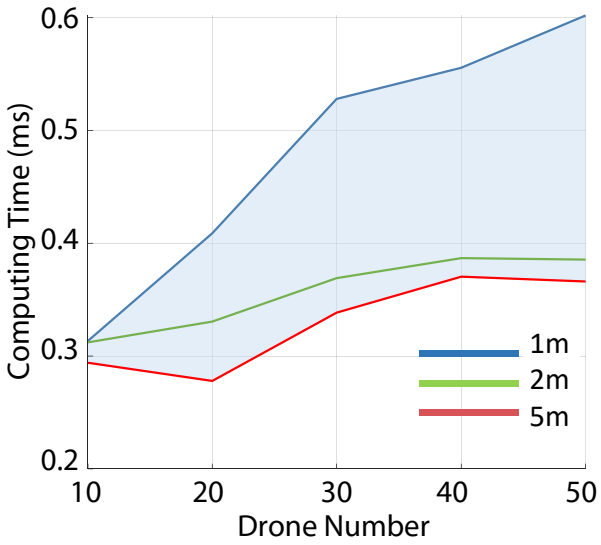

Fig. S1. Scalability on computation efficiency. This figure provides the computing time for different drone numbers under three swarm distributions shown in Fig. S15.

Wireless Communication Quality

Communication capacity can be affected by environmental signal interference, connection topology, and drone distribution. To control signal interference, we conduct this experiment in a quiet park with drones hovering and UWBs turned on. Then, in this test, we compare two topologies, namely a centralized network with one access point and a peer-to-peer ad hoc network using the hardware described in Section *Palm-Sized Drone Hardware*. In addition, various distributions illustrated in Fig. S2A from tight to loose, are tested. Two widely used metrics, bandwidth, and latency, are adopted to assess network capacity. Since we write reliable network communication software that guarantees data arrival, the packet loss rate is not evaluated (which equals either 0% or 100%). The bandwidth B_w is quantized to the highest trajectory (175 bytes each) broadcasting frequency by all the drones together before reaching the 250-ms latency threshold. Latency T_L is recorded at 5 Hz of the trajectory broadcasting rate.

To be collision-free, in a centralized network, each drone must maintain a direct connection to the access point (AP), so we record the worst value among all drones. By contrast, for the

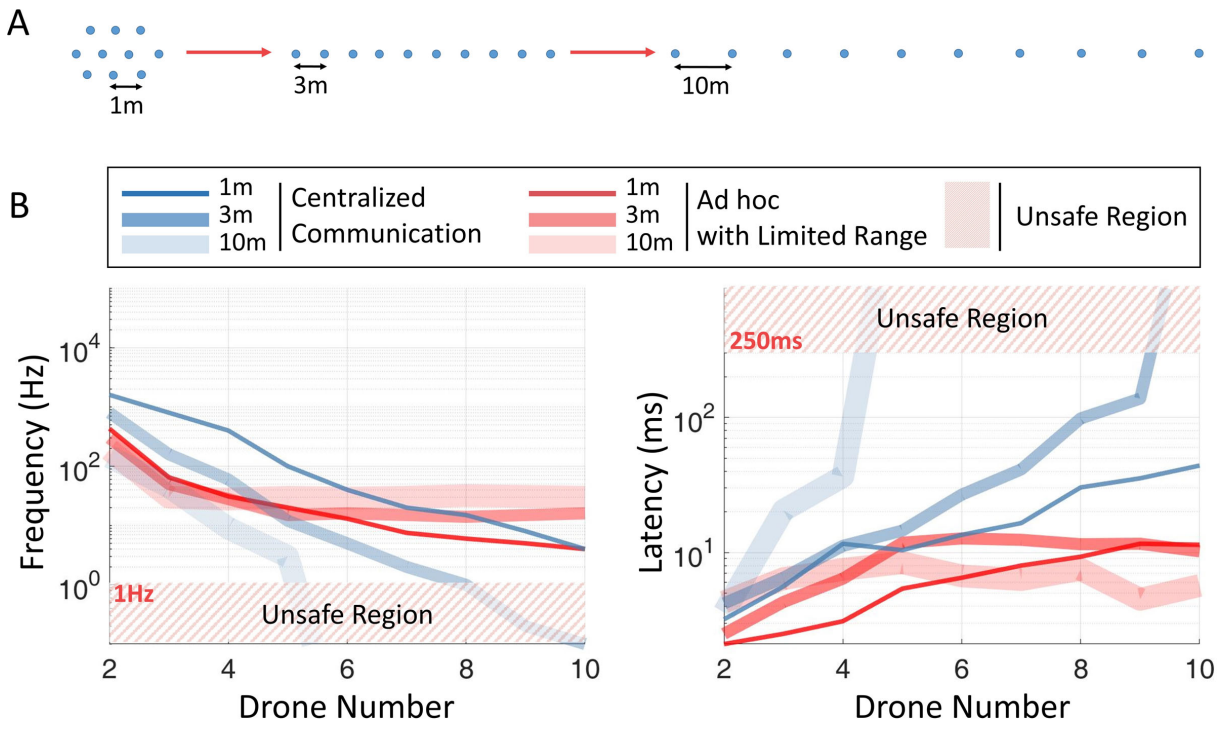

Fig. S2. Scalability on communication quality. (**A**) Three kinds of swarm distributions. (**B**) Network quality for two kind of topologies under three swarm distributions.

ad hoc network, each drone directly connects to the others. Furthermore, when two drones get further than twice the planning horizon from each other, the collision penalty between them is always zero, and freedom from collision is guaranteed naturally. Thus, we only evaluate data between each pair of drones with a relative distance below twice the planning horizon (15 m in this experiment) for an ad hoc network.

From the results in Fig. S2B, a centralized network performs better in tight formation, while an ad hoc network shows better communication quality when the formation is loose. From the integral view, an ad hoc network scales better with the number of drones when in loose distribution, while a centralized network provides higher bandwidth when the drone number is small; thus, both networks are complementary to an extent. In theory, as a peer-to-peer network, an ad hoc network creates connections that follow factorial complexity in a dense formation, while a

centralized network only follows linear complexity in this scenario. Therefore, developers must choose a proper network topology or adopt both types, of them according to task characteristics. The proposed drone platform supports both networks.

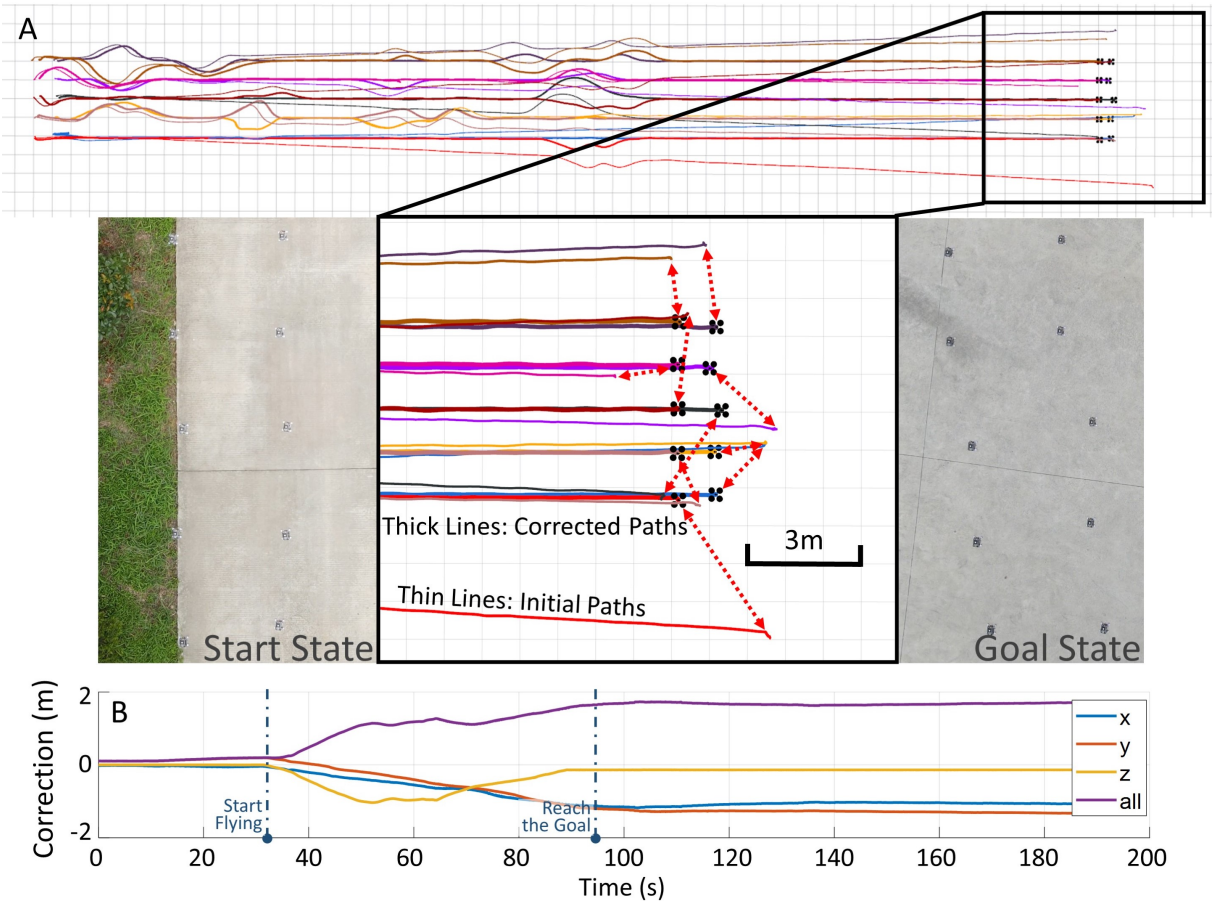

Fig. S3. Localization drift corrections. (**A**) Thin and thick lines are trajectories before and after corrections, respectively. The former ones show significant drifts that could have ruined the formation if not corrected. (**B**) Localization drift of drone 0. More drones are shown in Fig. S13.

Section S3. Assessments of Localization Drift Correction

The data recorded from the experiment in Section *Formation Navigation in the Wild* are given in Fig. S3. The desired start and goal shapes are 2×5 drone rectangles with 1-m spacings. With

the corrected localization, the drone managed to maintain the shape after a 55-m flight, although they are not precisely aligned in yaw at the start point. By contrast, Figs. S3 and S13 show the magnitude of the estimated drift, which reaches approximately 3 m. Under such significant drift, the formation suffers severe shape distortion and reciprocal collision if no correction is made. However, since no global information such as GPS data is available, accumulated errors for the entire team must exist. That is why we add some fixed anchors in Section *Intensive Reciprocal Avoidance Evaluation*. In summary, the necessity and effectiveness of the proposed correction method are validated here. The block diagram is shown in Fig. S14.

Section S4. Quantitative Analysis of the Gate-passing Test

To further analyze the planner performance, the gate-passing test in Section *Benchmark Comparisons* is conducted again with some representative data visualized in Fig. S4. Among the four sub-figures, B and C are mutually constrained. This means, for a shorter flight time, these drones must fly as fast as possible under the maximum speed of 2 m/s, while they also have to alter velocity to maintain a relative distance. Then, for drones in front, velocity can be increased to reduce flight time while enlarging relative distance, but the velocity is limited. For the drones left behind, reducing the velocity as little as possible to maintain the shortest relative distance to those in front is the best strategy. The above is exactly what is achieved in Fig. S4B and C, where drones managed to fly across the gate at the maximum velocity and minimum relative distance allowed. What's more, from Fig. S4D, we can conclude that drones fly along the shortest trajectory without detours or wasting of energy.

Section S5. Trajectory Parameterization for Aerial Robots

The pros and cons of typical trajectory parameterization for aerial robots are summarized in Table S2. **Polynomial** curves are widely used by traditional approaches (*4, 10, 38*), which mostly

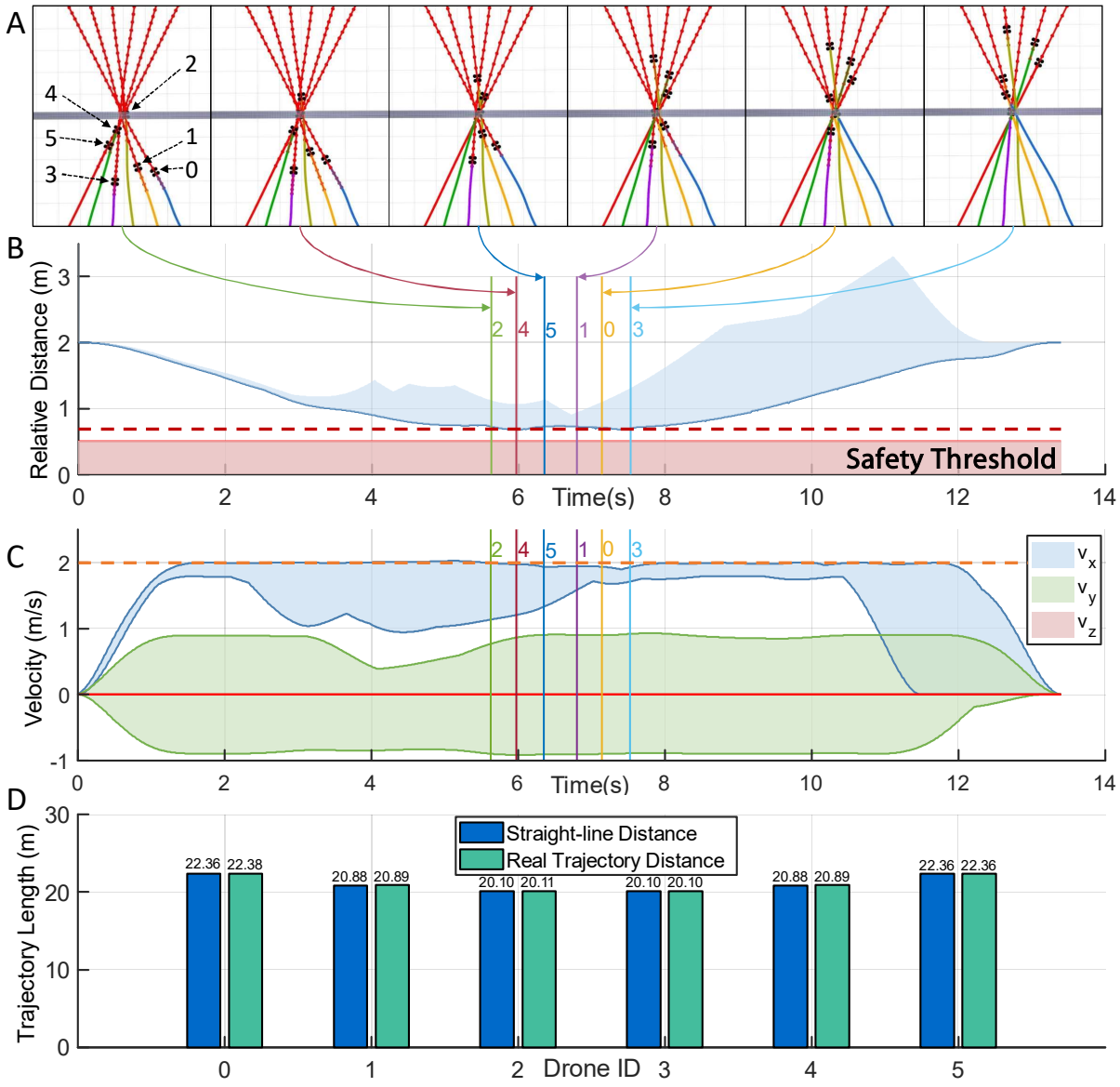

Fig. S4. Performance evaluation in the gate-passing test. (**A**) Six snapshots taken when each drone is passing the gate. (**B**) The minimum relative distances recorded by each drone are denoted by the light blue area, the lower bound of which is emphasized by a blue curve. Multicolored lines marked with numbers of drone ID indicate the time at which each drone flies through the gate. Red dotted line is the 0.7-m swarm clearance in optimization. (**C**) Minimum-to-maximum velocity curves. The orange dotted line at a 2-m/s velocity is the velocity limit. (**D**) Comparison between real trajectory lengths and straight-line distances (the minimum length in theory).

use polynomial coefficients as decision variables, along with explicit continuity constraints. Many works (*10, 38*) deform a piece-wise polynomial trajectory by adding intermediate waypoints. However, the distribution of waypoints requires a careful configuration to balance feasibility satisfaction and computational burden. To optimize the time profile of a polynomial, the representative method uses gradient descent with finite differences (*38*). Nevertheless, this method requires a dense matrix inversion to transform waypoints to coefficients, making the optimization quickly intractable when the problem scales up. **Bézier** and **B-Spline** share the *convex hull* nature, which indicates that the curve is entirely contained in the convex hull of its control points (*13, 23*). Therefore, these curves are convenient for adding constraints, which can be done by merely restricting the control points inside feasible convex regions. Many works (*24, 37, 50*) follow this underlying property and show successful applications in aerial swarms. However, this property indeed brings conservativeness, as analyzed by (*47*), preventing a trajectory from being aggressive near its physical limits. Moreover, an n order **B-Spline** is naturally $n-1$ order continuous (*48*). Thus no continuity constraint is required as long as the B-Spline has a qualified order. The time profile of a **Bézier** curve can be adjusted with acceptable complexity by its basis (*23*), while this approach is rather complicated for a B-Spline due to its time evaluation containing high nonlinearity. **MINCO** (*57*) is a newly developed trajectory representation specialized for integrator chain systems, and its core feature is that it efficiently handles a wide variety of constraints while retaining spatial and temporal optimality. By discretizing the trajectory to add constraints, MINCO is less conservative but requires further checks after obtaining the solution. Although MINCO is the most complicated representation and requires a sophisticated implementation, it provides a solid foundation for spatial-temporal drone swarm planning.

Table S2. Pros and cons of commonly used trajectory parameterizations.

Method	Continuity	Safety	Dynamical Feasibility	Temporal Optimization	Implementation Difficulty
Polynomial	Extra Overhead	By Discretization		Coupled	Easy
Bézier Curve		Analytical but Conservative			Medium
B-Spline	By Nature			Intractable	
MINCO		By Discretization		Decoupled	

Section S6. Spatial-Temporal Trajectory Optimization for Swarms — A More General and Detailed Problem Formulation

MINCO Trajectory Class

MINCO is a polynomial trajectory class defined as

$$\text{MINCO} = \Big\{ p(t) : [0, t_M] \mapsto \mathbb{R}^m \Big| \ \mathbf{c} = \mathcal{M}(\mathbf{q}, \mathbf{T}), \\ \mathbf{q} \in \mathbb{R}^{m \times M-1}, \ \mathbf{T} \in \mathbb{R}^M_{>0} \Big\},$$

where $p(t)$ is an m-dimensional M-piece polynomial trajectory with degree $N, N = 2s - 1$ with s the order of the relevant integrator chain, $\mathbf{c} = (\mathbf{c}_1^{\mathrm{T}}, \ldots, \mathbf{c}_M^{\mathrm{T}})^{\mathrm{T}} \in \mathbb{R}^{2Ms \times m}$ the polynomial coefficient with $\mathbf{c}_i \in \mathbb{R}^{2s \times m}$ the coefficient matrix, $\mathbf{q} = (\mathbf{q}_1, \ldots, \mathbf{q}_{M-1})$ the intermediate waypoints, $\mathbf{T} = (T_1, T_2, \ldots, T_M)^{\mathrm{T}}$ the time allocated for all pieces, $t_i = \sum_{j=1}^{i} T_j, \ i \in [1, M]$, and $t_0 = 0$ in the Supplementary Materials. The notion $\mathcal{M}(\mathbf{q}, \mathbf{T})$ equals $\mathcal{M}_{\mathbf{q},\mathbf{T}}$ in the main text. Specifically, the trajectory is evaluated as

$$p(t) = p_i(t - t_{i-1}), \ \forall t \in [t_{i-1}, t_i]. \tag{S1}$$

The i-th piece $p_i(t) : [0, T_i] \mapsto \mathbb{R}^m$ is defined by

$$p_i(t) = \mathbf{c}_i^{\mathrm{T}} \beta(t), \ \forall t \in [0, T_i], \tag{S2}$$

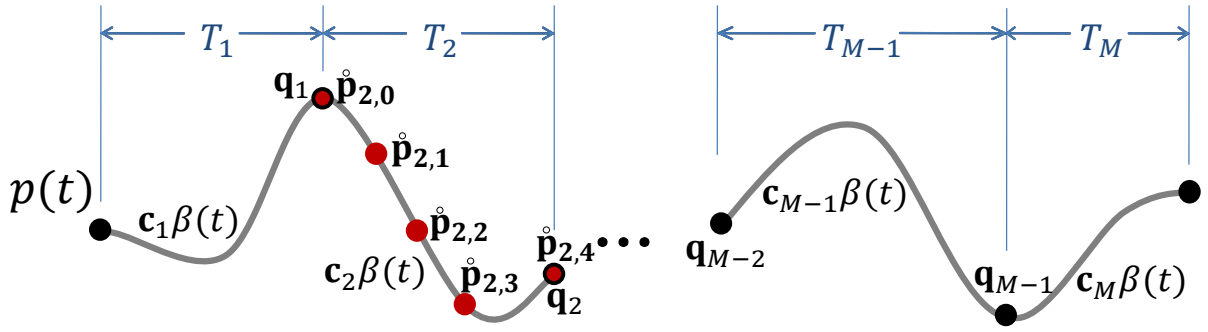

Fig. S5. Parameters of a MINCO trajectory and its constraint evaluation. The trajectory is directly parameterized by each waypoint $\mathbf{q}_i$ and time T_i. Red dots are *constraint points* $\mathring{\mathbf{p}}$ at which any constraint is evaluated and propagated to every $\mathbf{q}_i$ and T_i.

where $\beta(t) := [1, t, \cdots, t^N]^{\mathrm{T}}$ is the natural basis. The core of MINCO is the parameter mapping $\mathbf{c} = \mathcal{M}(\mathbf{q}, \mathbf{T})$ constructed from Theorem 2 in (*57*). Technically, the mapping directly constructs a minimum control trajectory for an m-dimensional s-integrator chain for any specified initial and terminal conditions. An instance in MINCO is intuitively shown in Fig. S5. Owing to the limited paper length, we only intuitively introduce the features of MINCO. Detailed proofs are presented in (*57*).

Feature 1: Compactly represented by $\mathbf{q}$ and $\mathbf{T}$, MINCO is a class of $\mathcal{C}^{s-1}$ polynomial trajectories satisfying the following control effort minimization:

$$\min_{p(t)} \int_{t_0}^{t_M} p^{(s)}(t)^{\mathrm{T}} \mathbf{W} p^{(s)}(t) \mathrm{d}t, \tag{S3a}$$

$$s.t.\ \ p^{[s-1]}(t_0) = \bar{p}_o,\ p^{[s-1]}(t_M) = \bar{p}_f, \tag{S3b}$$

$$p(t_i) = \mathbf{q}_i,\ 1 \le i < M, \tag{S3c}$$

$$t_{i-1} < t_i,\ 1 \le i \le M, \tag{S3d}$$

where $\mathbf{W} \in \mathbb{R}^{m \times m}$ is a diagonal matrix with positive entries, $p^{[s-1]} = [p, p^{(1)}, p^{(2)}, \cdots, p^{(s-1)}]$ with , $\bar{p}_o$ and $\bar{p}_f \in \mathbb{R}^{m \times s}$ the specified initial and terminal conditions, respectively. The trajectory is enforced to pass $\mathbf{q}_i$ at time t_i.

Feature 2: The evaluation and differentiation of the mapping $\mathbf{c} = \mathcal{M}(\mathbf{q}, \mathbf{T})$ benefits from

linear complexity. A more specific correspondence can be expressed as the following function

$$\mathbf{M}(\mathbf{T})\mathbf{c} = \mathbf{b}(\mathbf{q}), \tag{S4}$$

where $\mathbf{M}(\mathbf{T}) \in \mathbb{R}^{2Ms\times 2Ms}$ is a banded matrix with non-singularity for any $\mathbf{T} \succ \mathbf{0}$ ensured by Theorem 2 in (*57*), $\mathbf{b}(\mathbf{q}) \in \mathbb{R}^{2Ms\times m}$. The conversion between these two trajectory representations, i.e., $(\mathbf{c}, \mathbf{T})$ and $(\mathbf{q}, \mathbf{T})$ is achieved using *Banded PLU Factorization* with $O(M)$ linear time and space complexity. The evaluation is extremely fast, which takes approximately 1 μs per piece for minimum-jerk ($s = 3$) trajectory generation on a desktop CPU.

The formulation of $\mathbf{M}$ and $\mathbf{b}$ are

$$\mathbf{M} = \begin{pmatrix} \mathbf{F}_0 & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{E}_1 & \mathbf{F}_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{E}_2 & \mathbf{F}_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{F}_{M-1} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{E}_M \end{pmatrix}, \tag{S5a}$$

$$\mathbf{E}_i = (\beta^{(0)}(T_i), \beta^{(0)}(T_i), \beta^{(1)}(T_i), \cdots, \beta^{(2s-2)}(T_i))^{\mathrm{T}} \in \mathbb{R}^{2s\times 2s},\ i \in [1, M-1] \tag{S5b}$$

$$\mathbf{F}_i = (\mathbf{0}_{2s\times 1}, -\beta^{(0)}(0), \cdots, -\beta^{(2s-2)}(0))^{\mathrm{T}} \in \mathbb{R}^{2s\times 2s},\ i \in [1, M-1] \tag{S5c}$$

$$\mathbf{E}_M = \left(\beta^{(0)}(T_M), \cdots, \beta^{(s-1)}(T_M)\right)^{\mathrm{T}} \in \mathbb{R}^{s\times 2s}, \tag{S5d}$$

$$\mathbf{F}_0 = \left(\beta^{(0)}(0), \cdots, \beta^{(s-1)}(0)\right)^{\mathrm{T}} \in \mathbb{R}^{s\times 2s}; \tag{S5e}$$

$$\mathbf{b} = \left(\ \mathbf{D}_0,\ \ \mathbf{D}_1,\ \ \mathbf{0}_{m\times(2s-1)},\ \ \cdots,\ \ \mathbf{D}_{M-1},\ \ \mathbf{0}_{m\times(2s-1)},\ \ \mathbf{D}_M\ \right)^{\mathrm{T}}, \tag{S6a}$$

$$\mathbf{D}_i = \mathbf{q}_i \in \mathbb{R}^m,\ i \in [1, M-1], \tag{S6b}$$

$$\mathbf{D}_0 = \bar{p}_o,\ \mathbf{D}_M = \bar{p}_f,\ \mathbf{D}_0, \mathbf{D}_M \in \mathbb{R}^{m\times s}. \tag{S6c}$$

Feature 3: Feature 2 allows any user-defined objective or constraint function $F(\mathbf{c}, \mathbf{T})$ with available gradients applicable to MINCO parameterized in $(\mathbf{q}, \mathbf{T})$. Specifically, the correspond-

ing objective of MINCO is computed as

$$H(\mathbf{q},\mathbf{T}) = F(\mathcal{M}(\mathbf{q},\mathbf{T}),\mathbf{T}). \tag{S7}$$

Then the mapping $\mathbf{c} = \mathcal{M}(\mathbf{q},\mathbf{T})$ gives a linear-complexity way of computing $\partial H/\partial \mathbf{q}$ and $\partial H/\partial \mathbf{T}$ from the corresponding $\partial F/\partial \mathbf{c}$ and $\partial F/\partial \mathbf{T}$, i.e.,

$$\begin{array}{ccccccc} \frac{\partial H}{\partial \mathbf{q}} & = & \frac{\partial F}{\partial \mathbf{c}} & \frac{\partial \mathbf{c}}{\partial \mathbf{q}}, & \frac{\partial H}{\partial \mathbf{T}} & = & \frac{\partial F}{\partial \mathbf{c}}\frac{\partial \mathbf{c}}{\partial \mathbf{T}} + \frac{\partial F}{\partial \mathbf{T}}, \\ \uparrow & & \downarrow & \restriction & \uparrow & & \downarrow \quad \restriction \qquad \downarrow \end{array} \tag{S8}$$

where derivatives marked by $\uparrow$ are the gradients finally required by a numerical optimizer, $\downarrow$ are the elements calculated by user code, and $\restriction$ are just computed using the linear-complexity mapping $\mathbf{c} = \mathcal{M}(\mathbf{q},\mathbf{T})$, i.e., Eqs. S4 –S6. Users need not consider the constraints between $\mathbf{c}$ and $\mathbf{T}$ as if decoupled. Next, a high-level optimizer is able to optimize the objective efficiently.

Time Integral Constraints

Conventionally, requirements for dynamical feasibility and collision avoidance are formulated as functional-type constraints $\mathcal{G}(p(t),\dots,p^{(s)}(t)) \preceq \mathbf{0}, \forall t \in [0,t_M]$, $\mathcal{G} \in \mathbb{R}^{n_g}$ with n_g the number of types of constraints. However, this formulation cannot be directly handled by constrained optimization, since $\mathcal{G}$ contains infinitely many inequality constraints. Thus, we transform $\mathcal{G}$ into a finite-dimensional one via an integral of constraint violation. Practically, the integral is transformed into a weighted sum $J_\Sigma(\mathbf{c},\mathbf{T})$ of the sampled penalty function. In most cases where constraints are decoupled, i.e., constraints $\mathcal{G}(p^{[s]}(t))$ with $t_i \le t < t_{i+1}$ are merely determined by $\mathbf{c}_i$ and T_i, $p^{[s]} = [p, p^{(1)}, \cdots, p^{(s)}]$, the penalty for an i-th trajectory piece is computed as

$$J_i(\mathbf{c}_i,T_i) = \frac{T_i}{\kappa_i}\sum_{j=0}^{\kappa_i} \bar{\omega}_j \chi^{\mathrm{T}} \max\left(\mathcal{G}(\mathbf{c}_i,T_i,\frac{j}{\kappa_i}),\mathbf{0}\right)^3, \tag{S9}$$

where κ_i is the sample number on the i-th piece, $\chi \in \mathbb{R}^{n_g}_{\ge 0}$ a vector of penalty weights with appropriately large entries (several orders of magnitude larger than the smoothness penalty),

and $(\bar{\omega}_0, \bar{\omega}_1, \ldots, \bar{\omega}_{\kappa_i-1}, \bar{\omega}_{\kappa_i}) = (1/2, 1, \cdots, 1, 1/2)$ the quadrature coefficients following the trapezoidal rule. We define the points determined by $\{\mathbf{c}_i, T_i, j/\kappa_i\}$ as *constraint points* $\mathring{\mathbf{p}}_{i,j} := p_i((j/\kappa_i)T_i)$, with $p_i(t)$ the i-th polynomial piece. Then $\mathcal{G}(\mathring{\mathbf{p}}_{i,j}) := \mathcal{G}(\mathbf{c}_i, T_i, j/\kappa_i)$. Note that $\mathring{\mathbf{p}}_{i,\kappa_i} = \mathring{\mathbf{p}}_{i+1,0}$. The cubic penalty is used here for its twice continuous differentiability. $J_\Sigma(\mathbf{c}, \mathbf{T})$ is then defined as the summation:

$$J_\Sigma(\mathbf{c}, \mathbf{T}) = \sum_{i=1}^{M} J_i(\mathbf{c}_i, T_i). \tag{S10}$$

Since the gradients w.r.t. $(\mathbf{c}, \mathbf{T})$ are constructed from all $(\mathbf{c}_i, \mathbf{T}_i)$, we only give gradient templates for $\mathbf{c}_i$ and T_i using the chain rule when Eq. S9 holds,

$$\frac{\partial J_\Sigma}{\partial \mathbf{c}_i} = \frac{\partial J_i}{\partial \mathbf{c}_i} = \frac{\partial J_i}{\partial \mathcal{G}} \frac{\partial \mathcal{G}}{\partial \mathbf{c}_i}, \tag{S11}$$

$$\frac{\partial J_\Sigma}{\partial T_i} = \frac{\partial J_i}{\partial T_i} = \frac{J_i}{T_i} + \frac{\partial J_i}{\partial \mathcal{G}} \frac{\partial \mathcal{G}}{\partial t} \frac{\partial t}{\partial T_i}, \tag{S12}$$

$$\frac{\partial J_i}{\partial \mathcal{G}} = 3\frac{T_i}{\kappa_i} \sum_{j=0}^{\kappa_i} \bar{\omega}_j \chi^{\mathrm{T}} \max(\mathcal{G}, \mathbf{0})^2, \tag{S13}$$

$$t = \frac{j}{\kappa_i} T_i, \quad \frac{\partial t}{\partial T_i} = \frac{j}{\kappa_i}. \tag{S14}$$

From the above equations, Eqs. S11 –S14, once the gradients of constraints $\mathcal{G}$ relative to polynomial coefficients $\mathbf{c}_i$ and time t are evaluated, the gradients of $J_\Sigma(\mathbf{c}, \mathbf{T})$ are efficiently computed, which are then substituted into Eq. S8 to acquire final gradients respective to $\mathbf{q}$ and $\mathbf{T}$.

Penalty Formulation

The basic requirements on a desired trajectory are smoothness, total time, dynamical feasibility, as well as safety among obstacles and other agents. In this paper, we adopt MINCO to address above all concerns. Since MINCO is naturally guaranteed $\mathcal{C}^{s-1}$ by Theorem 2 in (*57*), no extra effort must be paid to trajectory continuity. The trajectory generation is formulated as an

unconstrained optimization problem:

$$\min_{\mathbf{q},\mathbf{T}} \sum_{x} \lambda_x J_x, \tag{S15}$$

where J_x are J_Σ instances, λ_x is the weight for each penalty, and subscripts $x = \{s, t, d, o, w, u\}$ represent smoothness (s), total time (t), dynamical feasibility (d), obstacle avoidance (o), swarm reciprocal avoidance (w), and uniform distribution of constraint points (u), respectively. More task-specific penalties can also be added. The problem is solved via standard, unconstrained nonlinear optimization.

Smoothness J_s

The smoothness metric follows the definition in Eq. S3b. Without loss of generality, considering a single dimension of the i-th piece $p_i(t) : [0, T_i] \mapsto \mathbb{R}$ of a polynomial spline, its derivatives with respect to $\mathbf{c}_i$ and T_i are

$$\frac{\partial J_s}{\partial \mathbf{c}_i} = 2\left(\int_0^{T_i} \beta^{(s)}(t)\beta^{(s)}(t)^{\mathrm{T}} dt\right)\mathbf{c}_i, \tag{S16}$$

$$\frac{\partial J_s}{\partial T_i} = \mathbf{c}_i^{\mathrm{T}}\beta^{(s)}(T_i)\beta^{(s)}(T_i)^{\mathrm{T}}\mathbf{c}_i. \tag{S17}$$

Total Time J_t

A shorter flight time is desirable, so we also minimize the weighted total execution time $J_t = \sum_{i=1}^{M} T_i$. Obviously, its gradients $\partial J_t/\partial \mathbf{c} = \mathbf{0}$, $\partial J_t/\partial \mathbf{T} = \mathbf{1}$.

Dynamical Feasibility Penalty J_d

For multicopters, dynamical feasibility is always guaranteed by limiting the trajectory's derivatives. In this paper, we limit the amplitude of velocity, acceleration, and jerk. Following Eq. S9, constraints of velocity, acceleration, and jerk are denoted by

$$\mathcal{G}_v = \dot{p}(t)^2 - v_m^2, \quad \mathcal{G}_a = \ddot{p}(t)^2 - a_m^2, \quad \mathcal{G}_j = \dddot{p}(t)^2 - j_m^2, \tag{S18}$$

where v_m, a_m, j_m are maximum allowed velocity, acceleration and jerk, respectively. The corresponding gradients are

$$\frac{\partial \mathcal{G}_x}{\partial \mathbf{c}_i} = 2\beta^{(n)}(t)p_i^{(n)}(t)^{\mathrm{T}}, \quad \frac{\partial \mathcal{G}_x}{\partial t} = 2\beta^{(n+1)}(t)^{\mathrm{T}}\mathbf{c}_i p_i^{(n)}(t), \tag{S19}$$

where $x = \{v, a, j\}$, $n = \{1, 2, 3\}$, and $t = jT_i/\kappa_i$, respectively. Substituting Eq. S18 into S9 and S19 into S11 & S12 obtains the penalty and the gradients of $\mathbf{c}_i$ and $\mathbf{T}_i$.

Obstacle Avoidance Penalty J_o

In this paper, we adopt the front-end of collision evaluation devised by (*14*). A brief description is given here for better understanding. In (*14*), the information for an unsafe trajectory to escape collision is extracted by comparing the unsafe initial trajectory with a collision-free guiding path. As depicted in Fig. S6, a fixed safe point $\mathbf{s}$ with a fixed safe vector $\mathbf{v}$ is generated and recorded by a corresponding key point $\mathbf{p}_{\text{key}}$ on the trajectory. Then, the distance of $\mathbf{p}_{\text{key}}$ to the obstacle that the $\{\mathbf{s}, \mathbf{v}\}$ pair generated is defined as

$$d_o(\mathbf{p}_{\text{key}}, \{\mathbf{s}, \mathbf{v}\}) = (\mathbf{p}_{\text{key}} - \mathbf{s})^{\mathrm{T}}\mathbf{v}. \tag{S20}$$

Using the distance information, the trajectory deforms iteratively during optimization. If $\mathbf{p}_{\text{key}}$ discovers a new obstacle, a new $\{\mathbf{s}, \mathbf{v}\}$ pair is stacked to $\mathbf{p}_{\text{key}}$'s records. Therefore each $\mathbf{p}_{\text{key}}$ may record several $\{\mathbf{s}, \mathbf{v}\}$ pairs. Note that each $\mathbf{p}_{\text{key}}$ generates and possesses $\{\mathbf{s}, \mathbf{v}\}$ pairs privately, and thus does not calculate its distance from other $\mathbf{p}_{\text{key}}$s' $\{\mathbf{s}, \mathbf{v}\}$ pairs.

In this paper, constraint points $\mathring{\mathbf{p}}_{i,j}$ with $1 \leq i \leq M, 1 \leq j \leq \kappa_i$ are selected as the key points $\mathbf{p}_{\text{key}}$. To enforce safety, we penalize distance to obstacles by less than a obstacle clearance $\mathcal{C}_o$. Therefore, the obstacle avoidance constraint is defined as

$$\mathcal{G}_o = (\cdots, \mathcal{G}_{o_k}(\mathring{\mathbf{p}}_{i,j}), \cdots)^{\mathrm{T}} \in \mathbb{R}^{N_{\mathbf{sv}}}, \tag{S21}$$

$$\mathcal{G}_{o_k}(\mathring{\mathbf{p}}_{i,j}) = \mathcal{C}_o - d_o(\mathring{\mathbf{p}}_{i,j}, \{\mathbf{s}, \mathbf{v}\}_k), \tag{S22}$$

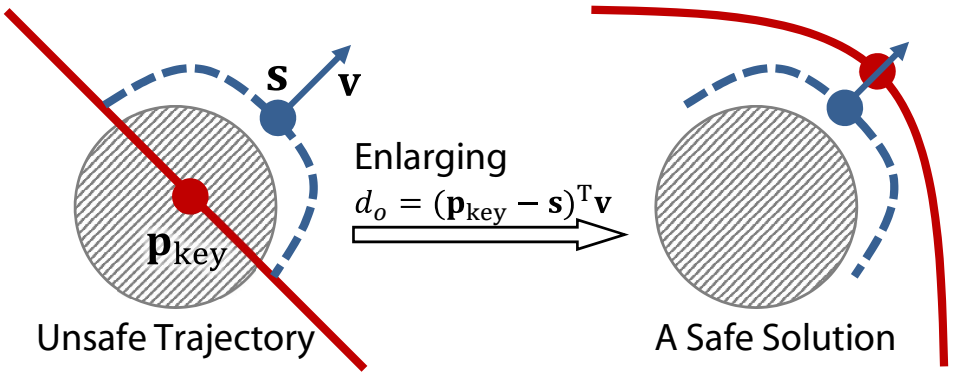

Fig. S6. Collision avoidance formulation. The red curve is the optimized trajectory and blue curve a collision-free path that starts and ends on the red one. A fixed point $\mathbf{s}$ with a vector $\mathbf{v}$ is generated to determine the collision penalty.

where $N_{\mathbf{sv}}$ is the number of $\{\mathbf{s}, \mathbf{v}\}$ pairs recorded by a single $\mathring{\mathbf{p}}_{i,j}$. For the case in which $d_o(\mathring{\mathbf{p}}_{i,j}, \{\mathbf{s}, \mathbf{v}\}_k) \leq \mathcal{C}_o$, the gradient is computed as

$$\frac{\partial \mathcal{G}_{o_k}}{\partial \mathbf{c}_i} = -\beta(t)\mathbf{v}^{\mathrm{T}}, \quad \frac{\partial \mathcal{G}_{o_k}}{\partial t} = -\dot{p}_i(t)^{\mathrm{T}}\mathbf{v}. \tag{S23}$$

Otherwise, $\partial \mathcal{G}_{o_k}/\partial \mathbf{c}_i = \mathbf{0}_{2s\times m}, \partial \mathcal{G}_{o_k}/\partial t = 0$.

Swarm Reciprocal Avoidance Penalty J_w

In this paper, a decentralized, non-cooperative swarm framework is adopted, which means that one agent receives other agents' trajectories as constraints and generates trajectories only for itself. Considering the u-th agent in a multicopter swarm containing U agents, swarm collision avoidance is guaranteed when the trajectory ${}^{u}p(t)$ of agent u maintains a distance greater than a safe clearance $\mathcal{C}_w$ to all the trajectories at the same global timestamps of other agents as depicted in Fig. S7. However, caution must be taken because we have always been using a relative time $t = jT_i/\kappa_i$ for $p_i(t)$ in our optimization, while the other agents' trajectories take an absolute timestamp $\tau = t_{of} + T_1 + \cdots + T_{i-1} + jT_i/\kappa_i$, with $t_{of} = t_w - \tau_w$ as a time offset to align t and τ from different world time t_w and τ_w, respectively. This indeed makes the penalty evaluated on the i-th piece depend on all its preceding piece times. We denote by

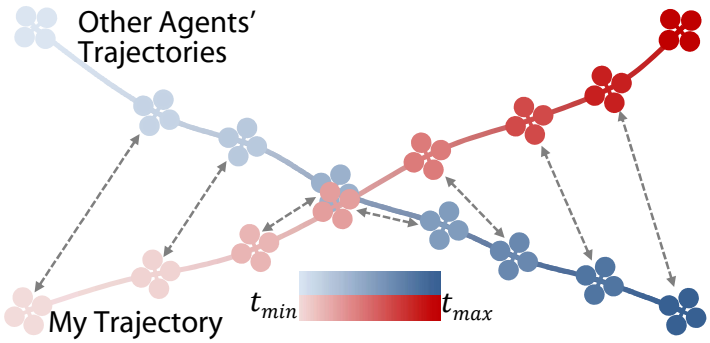

Fig. S7. Reciprocal collision avoidance formulation. Gray dotted arrows indicate states at the same absolute time, at which a safe distance must be ensured.

$G_w^{i,j}$ the constraint evaluated at the j-th constraint point on the i-th piece of ${}^u p(t)$, i.e., ${}^u p_i(t)$. Therefore the reciprocal avoidance constraint is defined as

$$\mathcal{G}_w^{i,j}({}^u p_i(t), \tau) = (\cdots, \mathcal{G}_{w_k}({}^u p_i(t), \tau), \cdots)^{\mathrm{T}} \in \mathbb{R}^{U-1}, k \neq u, \tag{S24}$$

$$\mathcal{G}_{w_k}^{i,j}({}^u p_i(t), \tau) = \mathcal{C}_w^2 - d_w^2({}^u p_i(t), {}^k p(\tau)), \tag{S25}$$

$$d_w({}^u p_i(t), {}^k p(\tau)) = \left\| \mathbf{E}^{1/2}({}^u p_i(t) - {}^k p(\tau)) \right\|, \tag{S26}$$

where ${}^k p(\tau)$ is the trajectory of the k-th agent that the u-th agent must avoid, $k \neq u$. The matrix $\mathbf{E} := \mathrm{diag}(1, 1, 1/c)$ with $c > 1$ transforms Euclidean distance into *ellipsoidal distance* with the minor axes at the z-axis to relieve the downwash risk from rotors. Squared distance is used to avoid *square root* operations.

When $d_w^2({}^u p_i(t), {}^k p(\tau)) \leq \mathcal{C}_w^2$, the gradient to $\mathbf{c}_i$ is

$$\frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial \mathbf{c}_i} = -2\beta(t)({}^u p_i(t) - {}^k p(\tau))^{\mathrm{T}} \mathbf{E}. \tag{S27}$$

The gradient to $\mathbf{T}$ is more complicated, since τ is a function of not only T_i, but T_1 to T_{i-1} as well when calculating the gradients for $\mathcal{G}_{w_k}^{i,j}$. This led to the gradient-propagate template Eq. S12 not holding here for $\partial J_\Sigma / \partial T_i \neq \partial J_i / \partial T_i$. When $d_w^2({}^u p_i(t), {}^k p(\tau)) \leq \mathcal{C}_w^2$, considering both preceding and current time profiles T_l for $1 \leq l \leq i$, the proper gradient should be computed as

$$\frac{\partial J_w}{\partial T_l} = \sum_{k=1,k\neq u}^{U} \frac{\partial J_{w_k}}{\partial T_l} = \sum_{k=1,k\neq u}^{U} \sum_{i=l}^{M} \sum_{j=1}^{\kappa_i} \frac{\partial J_{w_k}^{i,j}}{\partial T_l}, \tag{S28}$$

$$\frac{\partial J_{w_k}^{i,j}}{\partial T_l} = \frac{J_{w_k}^{i,j}}{T_l} + \frac{\partial J_{w_k}^{i,j}}{\partial \mathcal{G}_{w_k}^{i,j}} \frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial T_l}, \tag{S29}$$

$$\frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial T_l} = \frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial t} \frac{\partial t}{\partial T_l} + \frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial \tau} \frac{\partial \tau}{\partial T_l}, \tag{S30}$$

$$\frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial t} = 2\left({}^{k}p(\tau) - {}^{u}p_i(t)\right)^{\mathrm{T}} \mathbf{E}^{u}\dot{p}_i(t), \tag{S31}$$

$$\frac{\partial \mathcal{G}_{w_k}^{i,j}}{\partial \tau} = 2\left({}^{u}p_i(t) - {}^{k}p(\tau)\right)^{\mathrm{T}} \mathbf{E}^{k}\dot{p}(\tau), \tag{S32}$$

$$\frac{\partial t}{\partial T_l} = \begin{cases} \frac{j}{\kappa_i} & l = i, \\ 0 & l < i, \end{cases} \qquad \frac{\partial \tau}{\partial T_l} = \begin{cases} \frac{j}{\kappa_i} & l = i, \\ 1 & l < i. \end{cases} \tag{S33}$$

The trajectory ${}^{k}p(\tau)$ for any other agent is already known, and thus its derivatives are all available. $\partial J_{w_k}^{i,j}/\partial \mathcal{G}_{w_k}^{i,j}$ in Eq. S29 is computed following Eq. S13, while $(\partial \mathcal{G}/\partial t)(\partial t/\partial T_i)$ in Eq. S12 is replaced by Eq. S30. Next $\partial J_w/\partial \mathbf{T} = [\partial J_w/\partial T_1, \cdots, \partial J_w/\partial T_M]^{\mathrm{T}}$.

Uniform Distribution Penalty J_u

This penalty is not mentioned in the main body of the article since this well-packed module will not bring about sufficient insights to general readers in terms of understanding the methodology, but only some confusion. The purpose of this penalty is to make the constraint points $\mathring{\mathbf{p}}_{i,j}$ equally spaced for all $1 \leq i \leq M$ and $0 < j \leq \kappa_i$, which is a simpler substitution to the space-uniform variant of MINCO mentioned in (*57*). It is necessary since if all MINCO parameters $\mathbf{q}$ and $\mathbf{T}$ are optimized freely, some pieces of the trajectory tend to vanish to reduce the total cost. This phenomenon is harmful for three reasons. First, $T_i = 0$ for the i-th trajectory piece is a unique singular point for MINCO, which does not consider consecutive aligned waypoints. Second, since the spatial collision avoidance is constrained by a finite number of constraint points, non-uniform distribution increases the possibility of skipping some tiny or thin obstacles, as shown in Fig. S8. Third, since we compute the distance to obstacles following Eq. S20, the

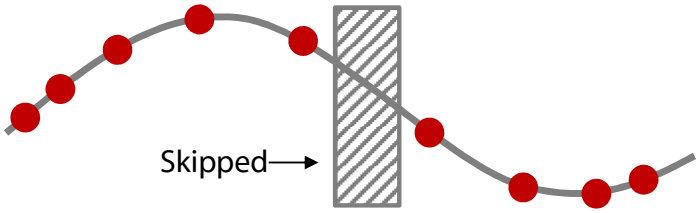

Fig. S8. Non-uniform constraint points (red dots) fail to detect a thin obstacle.

obstacles are modeled as planes. The accuracy of this modeling decreases when constraint points move along the plane and the uniform distribution reduces such movement.

To enforce the uniform distribution of constraint points, we penalize the variance of the squared distances between each pair of adjacent constraint points. For simplicity, we denote $N_c = \sum_{i=1}^{M} \kappa_i$ as the total number of distances. The variance is computed as

$$J_u = \mathrm{E}\left[\mathbf{R}^2\right] - \mathrm{E}\left[\mathbf{R}\right]^2 = \frac{1}{N_c}\left\|\mathbf{R}\right\|_2^2 - \frac{1}{N_c^2}\left\|\mathbf{R}\right\|_1^2, \tag{S34}$$

where $\mathbf{R} \in \mathbb{R}^{N_c}$ is defined by

$$\mathbf{R} = \left(\left\|\mathring{\mathbf{p}}_{1,1} - \mathring{\mathbf{p}}_{1,0}\right\|_2^2, \cdots, \left\|\mathring{\mathbf{p}}_{M,\kappa_M} - \mathring{\mathbf{p}}_{M,\kappa_M-1}\right\|_2^2\right). \tag{S35}$$

$\mathbf{R}$ is the squared distance vector for all $N_c + 1$ constraint points. We denote by $\mathbf{R}_k$ the k-th entry in $\mathbf{R}$, $k = j + \sum_{l=1}^{i-1} \kappa_l$. The gradient is computed as

$$\frac{\partial J_u}{\partial \mathbf{c}_i} = \sum_{j=1}^{\kappa_i} \beta\left(\frac{jT_i}{\kappa_i}\right)\left(\frac{\partial J_u}{\partial \mathring{\mathbf{p}}_{i,j}}\right)^{\mathrm{T}}, \tag{S36}$$

$$\begin{aligned}\frac{\partial J_u}{\partial \mathring{\mathbf{p}}_{i,j}} = {} & \frac{4}{N_c}\left(\mathbf{R}_{k-1} - \frac{\left\|\mathbf{R}\right\|_1}{N_c}\right)\left(\mathring{\mathbf{p}}_{i,j} - \mathring{\mathbf{p}}_{i,j-1}\right) \\ & - \frac{4}{N_c}\left(\mathbf{R}_{k} - \frac{\left\|\mathbf{R}\right\|_1}{N_c}\right)\left(\mathring{\mathbf{p}}_{i,j+1} - \mathring{\mathbf{p}}_{i,j}\right),\end{aligned} \tag{S37}$$

$$\frac{\partial J_u}{\partial T_i} = \frac{1}{\kappa_i}\sum_{j=1}^{\kappa_i} j\left(\frac{\partial J_u}{\partial \mathring{\mathbf{p}}_{i,j}}\right)^{\mathrm{T}} \mathbf{c}_i^{\mathrm{T}} \dot{\beta}\left(\frac{jT_i}{\kappa_i}\right). \tag{S38}$$

Temporal Constraint Elimination

An open-domain constraint is $\mathbf{T} \succ \mathbf{0}$, which is directly eliminated by a diffeomorphism map as is done in (*57*):

$$T_i = e^{\tau_i}, \tag{S39}$$

where τ_i is the unconstrained virtual time. Gradients w.r.t. T_i are propagated to τ_i following $\partial J/\partial \tau_i = (\partial J/\partial T_i)e^{\tau_i}$. More generally, any twice continuously differentiable bijection $f : \mathbb{R} \mapsto \mathbb{R}_{>0}$ suffices for this map.

Section S7. Collision Avoidance for Heterogeneous Swarms

In the conducted experiments, desired swarm clearance $\mathcal{C}_w$ is fixed due to identical agent size. If implementing the proposed planning algorithm for a heterogeneous swarm, each agent should broadcast its desired clearance, which is typically its radius plus a tolerance distance. Then, for the u-th agent that is going to avoid agent k, the swarm clearance between them is defined as ${}^{uk}\mathcal{C}_w = {}^{u}\mathcal{C}_w + {}^{k}\mathcal{C}_w$, where ${}^{u}\mathcal{C}_w$ and ${}^{k}\mathcal{C}_w$ are the desired clearances by agents u and k, respectively.

Section S8. Drone Removal on Depth Images

In the proposed system, an occupancy grid is adopted for static map representation, which can be affected by moving agents in depth images. Therefore, pixels of other agents are set to unknown. This procedure is performed at Line 14 in Alg. 1. The visual-inertial odometry that we utilize for localization requires grayscale images of static environments, which indeed contain moving agents. Nonetheless, the outlier is eliminated by RANSAC. Thus, moving agents are unlikely to affect our localization module which uses a wide angle-of-view camera.

Algorithm 1: DroneDetection$(\mathbf{u}, \mathbf{k})$

Notation: Detector $\mathbf{u}$, Target Drone $\mathbf{k}$,
Detector Pose $\mathbf{P_u}$, Target Pose $\mathbf{P_k}$,
Depth Image $\mathbf{I_u}$, Threshold $\mathcal{T}$.
Input: $\mathbf{I_u}, \mathbf{P_u}, \mathbf{P_k}$
Output: $\mathbf{CorrectP_k}, \mathbf{ModifiedI_u}$
begin
  $\mathbf{PriorP_k} \leftarrow PoseWorld2Cam(\mathbf{P_k}, \mathbf{P_u})$
  $\mathbf{PriorPixel_k} \leftarrow Pose2Pixel(\mathbf{PriorP_k})$
  if $InDepthImage(\mathbf{PriorPixel_k}, \mathbf{I_u})$ **then**
    $\mathbf{Box} \leftarrow BoundingBox(\mathbf{PriorPixel_k}, \mathbf{I_u}, \mathbf{PriorP_k})$
    for $\mathbf{Pixel}$ **in** $\mathbf{Box}$ **do**
      $\mathbf{P} \leftarrow Pixel2Pose(\mathbf{Pixel})$
      if $Distance(\mathbf{P}, \mathbf{P_k}) \leq \mathcal{T}$ **then**
        $\mathbf{DronePixels_k}.PushBack(\mathbf{Pixel})$
    $\mathbf{CorrectPixel_k} \leftarrow GetCenter(\mathbf{DronePixels_k})$
    $\mathbf{CorrectP_k} \leftarrow Pixel2Pose(\mathbf{CorrectPixel_k})$
    $\mathbf{ModifiedI_u} \leftarrow SetUnknown(\mathbf{I_u}, \mathbf{DronePixels_k})$
  return;

Drone Detection

Unlike (*65*), in which drone detection is done merely by neural networks, we propose a lightweight but robust approach by pixel gathering from depth images using agent position as an a priori. The algorithm is explained in Alg. 1. Function *PoseWorld2Cam()* transforms $\mathbf{P_k}$ into the three-dimensional (3D) camera frame of the detector $\mathbf{u}$. Function *Pose2Pixel()* transforms $\mathbf{PriorP_k}$ into the 2D camera frame represented by the conventional $u - v$ pixel axis of the detector $\mathbf{u}$. Function *InDepthImage()* returns whether the pixel value of $\mathbf{PriorPixel_k}$ is within the boundary of the image $\mathbf{I_u}$. Function *BoundingBox()* creates a bounding box centered at $\mathbf{PriorPixel_k}$ with size larger if the z-component of $\mathbf{PriorP_k}$ on $\mathbf{I_u}$ is smaller. Function *Pixel2Pose()* transforms $\mathbf{Pixel}$ with depth from $\mathbf{I_u}$ into the 3D world frame. Function *Distance()* calculates Euclidean distance between $\mathbf{P}$ and $\mathbf{P_k}$. Function *PushBack()* means data saving. Function *Get-*

Center() computes the average position of all the $\mathbf{DronePixels_k}$. Function *SetUnknown()* is an image operator that modifies the depth pixel values belonging to the detected drone to the unknown, and then the witnessed drone is removed. The $\mathbf{CorrectP_k}$ can be further used to correct localization drift.

Section S9. Failure Cases and Troubleshooting

Although a great deal of engineering considerations have been taken into account, limited by our knowledge and equipment, failures still occasionally occur. To those who are intended to reproduce our work to carry out their own research, possible solutions to some cases of failure are listed in Table S3.

Table S3. Cases of failure and troubleshooting.

Failure Cases	Frequency	Possible Solutions
Arm denied by FCU.	Often if using ekf2 estimator in PX4.	Change the default ekf2 to Q estimator (complementary filter) in PX4 Autopilot.
Initial state estimation not stable.	Becomes common after a crash.	Re-calibrate extrinsic parameters between camera and IMU.
RealSense D430 failed to boot.	Often if the power supply is insufficient.	Make sure the type-c port has enough power and use a high-current type-c cable.
RealSense D430 outputs significantly wrong distance measurements.	Rare.	This happens when pointing the camera at a highlighted plane or a surface with repeated textures; therefore, try to add some features on these surfaces.
Some of the images are corrupted when in flight.	Rare.	Caused by signal interfering or vibration, so check the connection of the camera cable and place it away from the battery, ESC, and motors.
Trajectory message loss.	Sometimes.	Use better network hardware and do not print all the information to the ground computer via network.
Emergency stops triggered frequently in dense environments.	Sometimes.	Increase the map resolution and set appropriate inflation; otherwise, some small gaps frequently occur and continuously switch between free and occupied.
Two drones collide.	Rare.	Caused by the poor network every time and meanwhile, two drones did not see each other in depth images, so fix the network first and consider enlarging the depth FOV (field of view).
Hitting an obstacle.	Rare.	RealSense D430 is the culprit every time, so check what goes wrong with this device following the above troubleshooting regarding it.

Section S10. More Discussion on Communication

The requirements for communication are task-specific. In experiments *Fly Through Dense Forest*, the goal positions have been written into configuration files before takeoff. Then, trajec-

tory is the only message type to be transmitted for inter-robot collision avoidance and relative localization correction. In such cases, the communication range must be at least twice the planning distance, since drones with spacing closer than this threshold will affect each other when planning the trajectory. If their distance is longer than this threshold, they are sure to be collision-free in a short period of future time without any avoidance actions. Benefiting from the developments of wireless communication technology, such a dozens-of-meters communication range has been achieved by most solutions, e.g., WiFi, UWB, and Zigbee.

In formation flight tasks, except for reciprocal avoidance, drones have to dynamically maintain a formation according to other drones' real-time positions extracted from received trajectories. This requires that each drone maintains at least one connection to others and the whole formation is inseparable regarding to wireless connection. Then adopting some multihop and routing protocols, the trajectory messages can reach to all other drones. In the experiment *Formation Navigation in the Wild*, the formation scale is relatively compact, therefore messages are directly shared in the whole swarm.

In the experiment *Intensive Reciprocal Avoidance Evaluation*, trajectory-transmission requirements are the same as those in *Fly Through Dense Forest*. However, the goals in this experiment are given by a ground computer in real-time. Therefore, a connection from the ground computer to each drone is necessary. In addition, the ground computer can be connected directly as in our experiments or through wireless routing.

In the experiment *Multi-Drone Tracking with Target Occlusion*, each drone combines goals from its own observations with others' observations received from the network. Therefore, more sharing means higher robustness to occlusion. If there is no observation from others, the drone can still track the target, but may lose it when encountering a significant occlusion. Thus, it is difficult to determine an exact threshold of communication range in this task. In our experiment, we share the observation among all drones.

Regarding communication bandwidth, in the bamboo forest experiment of 10 drones, the size of one trajectory is approximately 170 bytes. For the 65-m flight in the bamboo forest there are about 100 trajectories sent and 1000 trajectories received for each drone. The data rate (bandwidth) containing both sent and received information is approximately 2 kB/s and reached 8 kB/s at peak. These are minor values for WiFi network, which always provides a several megabytes-per-second (MBps) data rate. The statistics are listed in Table S4 and S5.

Table S4. Data sent in the bamboo forest flight. Trajectory is the only message type transmitted via network. **Traj.**: Trajectory; **Min.**: Minimal; **Max.**: Maximal; **Avg.**: Average.

Drone ID	Traj. Number	Total Size (bytes)	Min. Traj. Size (bytes)	Max. Traj. Size (bytes)	Avg. One Traj. Size (bytes)	Peak Data Rate (bytes/s)	Avg. Data Rate (bytes/s)
0	93	15811	127	175	170.01	875	171.63
1	96	16224	127	175	169.00	636	176.11
2	88	15064	127	175	171.18	525	163.52
3	98	16830	127	175	171.73	700	182.69
4	102	17418	127	175	170.76	1209	189.07
5	132	21628	127	175	163.85	2159	234.77
6	125	20755	127	175	166.04	1002	225.30
7	112	19088	127	175	170.43	2100	207.20
8	102	17514	127	175	171.71	700	190.12
9	93	15891	127	175	170.87	684	172.50
Avg.	**104.10**	**17622.30**	**127.00**	**175.00**	**169.56**	**1059.00**	**191.29**

Table S5. Data received in the bamboo forest flight. Trajectory is the only message type transmitted via network. **Traj.**: Trajectory; **Min.**: Minimal; **Max.**: Maximal; **Avg.**: Average.

Drone ID	Traj. Number	Total Size (bytes)	Min. Traj. Size (bytes)	Max. Traj. Size (bytes)	Avg. One Traj. Size (bytes)	Peak Data Rate (bytes/s)	Avg. Data Rate (bytes/s)
0	948	160412	127	175	169.21	6202	1741.29
1	945	159999	127	175	169.31	5377	1736.80
2	953	161159	127	175	169.11	5838	1749.40
3	943	159393	127	175	169.03	5377	1730.22
4	939	158805	127	175	169.12	5043	1723.84
5	909	154595	127	175	170.07	4295	1678.14
6	916	155468	127	175	169.72	5377	1687.62
7	929	157135	127	175	169.14	4502	1705.71
8	939	158709	127	175	169.02	6538	1722.80
9	948	160332	127	175	169.13	5377	1740.42
Avg.	**936.90**	**158600.70**	**127.00**	**175.00**	**169.29**	**5392.60**	**1721.62**

Figures

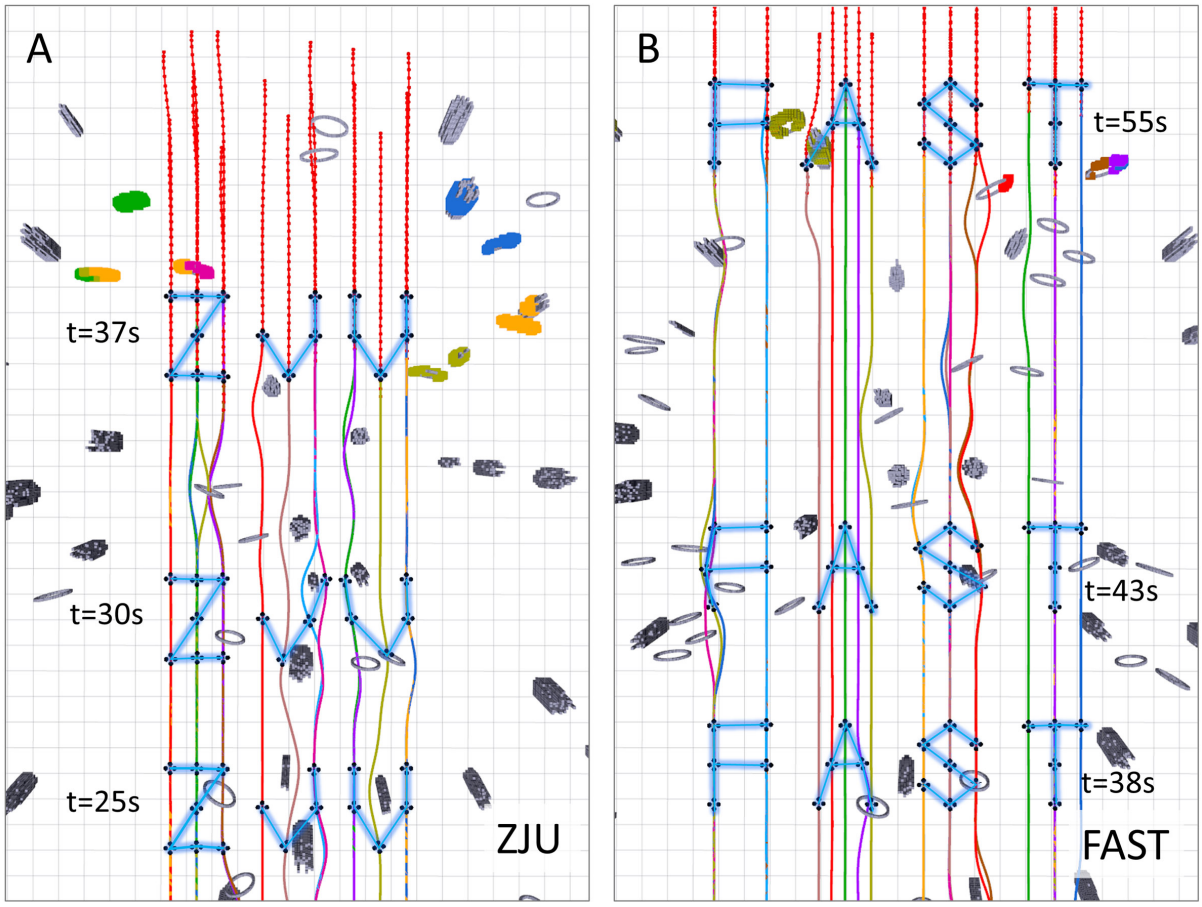

Fig. S9. Composite images of two formation flights. (**A**) Letters ZJU (ZheJiang University) formed by 16 drones. (**B**) Letters FAST (FAST Lab) formed by 22 drones.

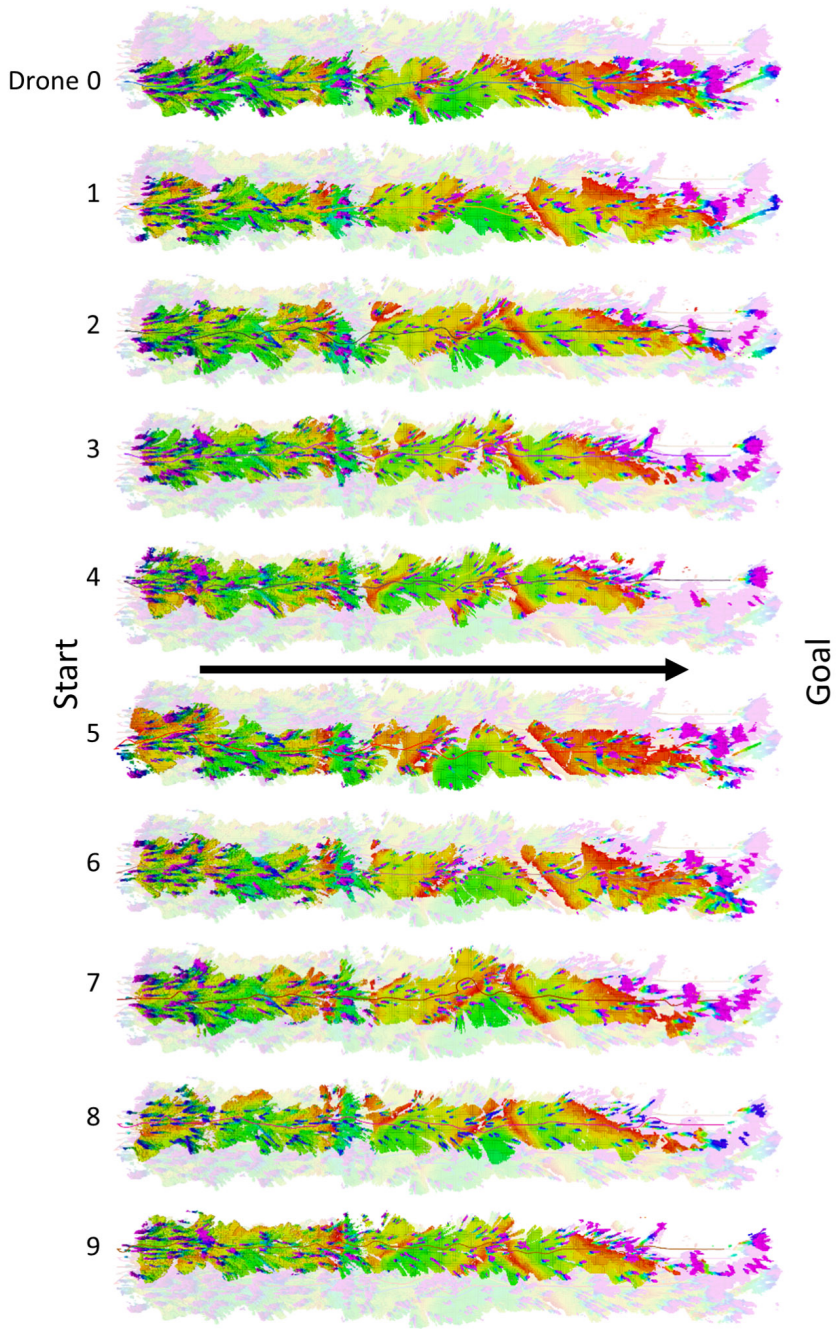

Fig. S10. Maps of the Bamboo Forest Built by Drones. The light-colored background denotes the map fused by overlaying 10 position-aligned drones' sub-maps. Different colors represent different heights. For example, green indicates the ground while the purple indicates bamboos. All trajectories are collision-free. Owing to the drift correction, 10 maps are all accurately aligned here.

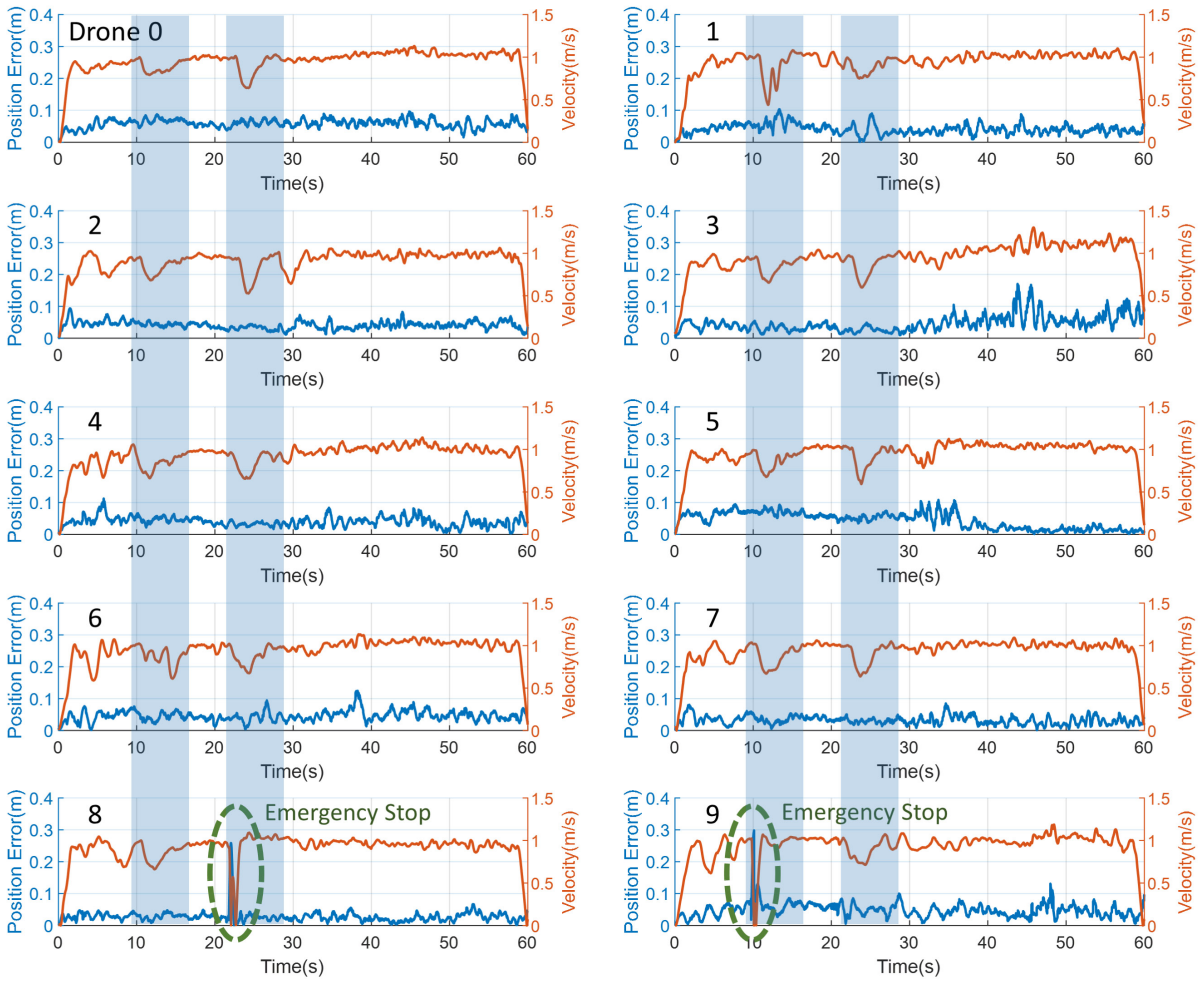

Fig. S11. Control errors during flights in Section *Formation Navigation in the Wild*. Green dotted ellipses indicate emergency stops activated by the safety check introduced in Section *Hierarchical Safety Guarantee*. Blue translucent areas indicate significant changes in speeds. Here, it is emergency stop rather than speed change that increases the position error.

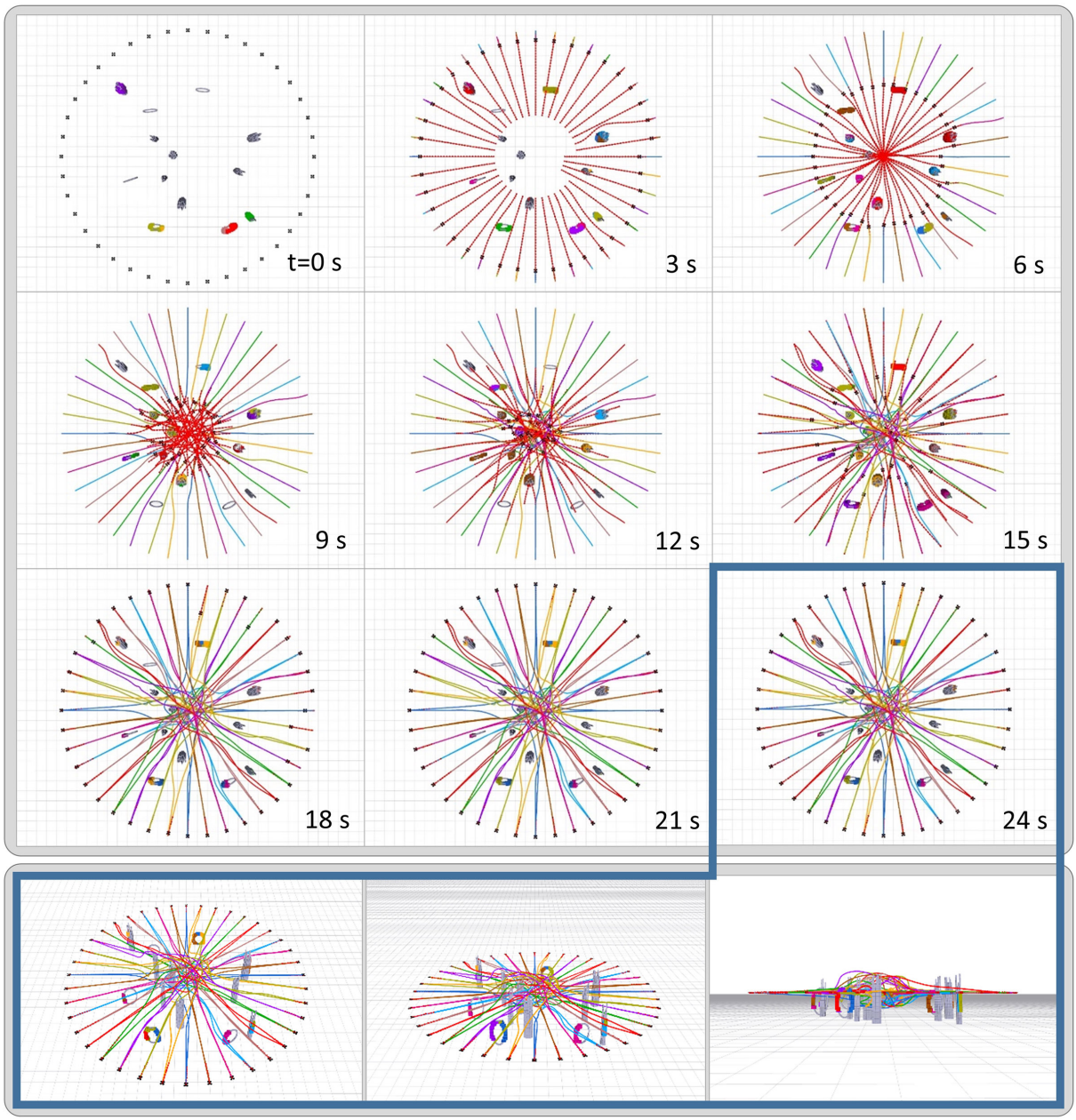

Fig. S12. A 40-drone position swap on a circle.

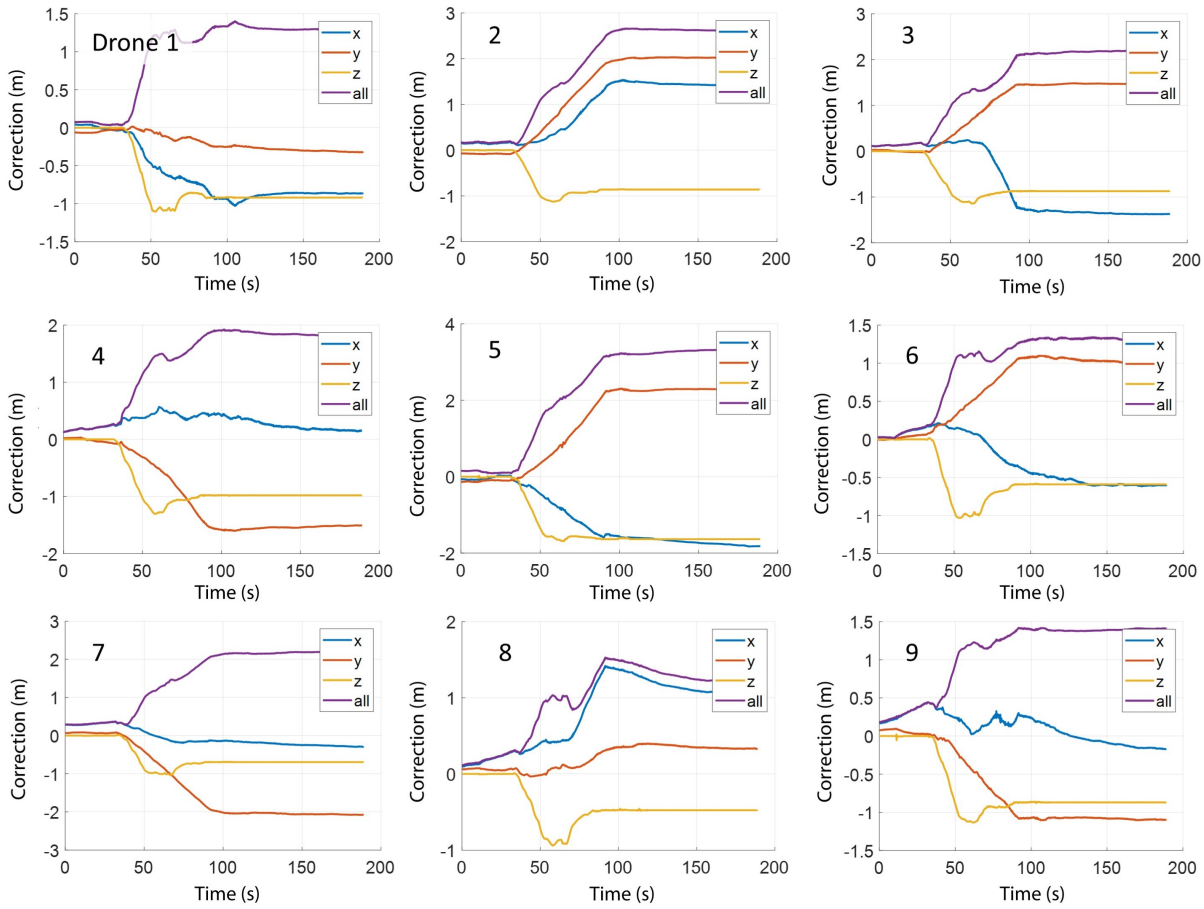

Fig. S13. Localization drift corrections for drones 1–9. Drone 0 is visualized in Fig. S3B.

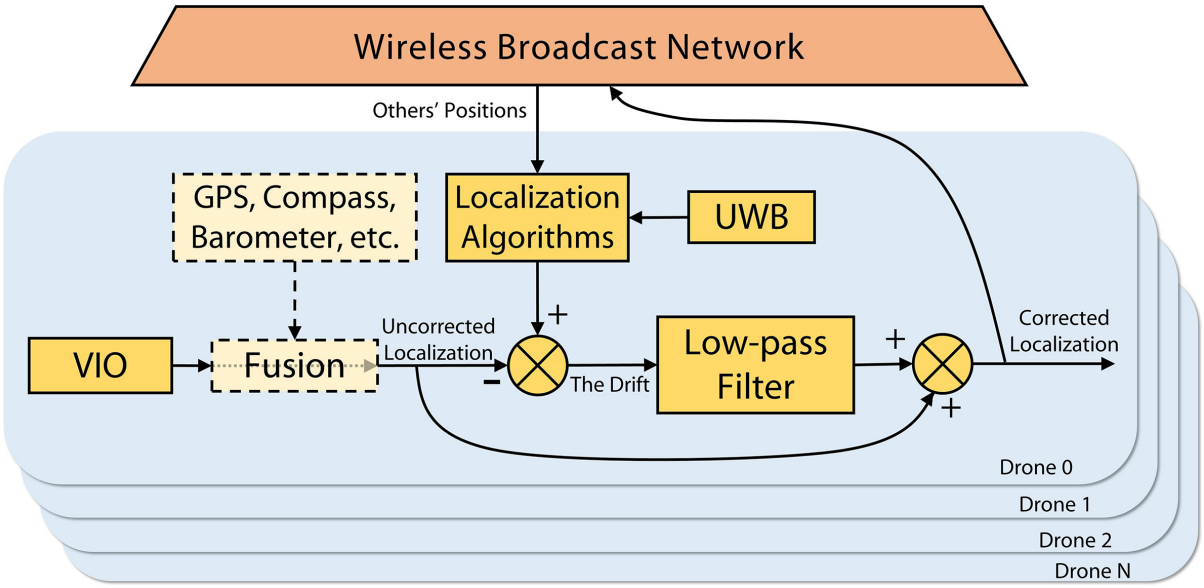

Fig. S14. Block diagram of localization drift correction. See Section *Localization and Drift Correction* for details. Sensors with corresponding fusion methods in dashed light-yellow boxes are optional and not adopted in our experiments. Positions in our framework are shared via trajectories.

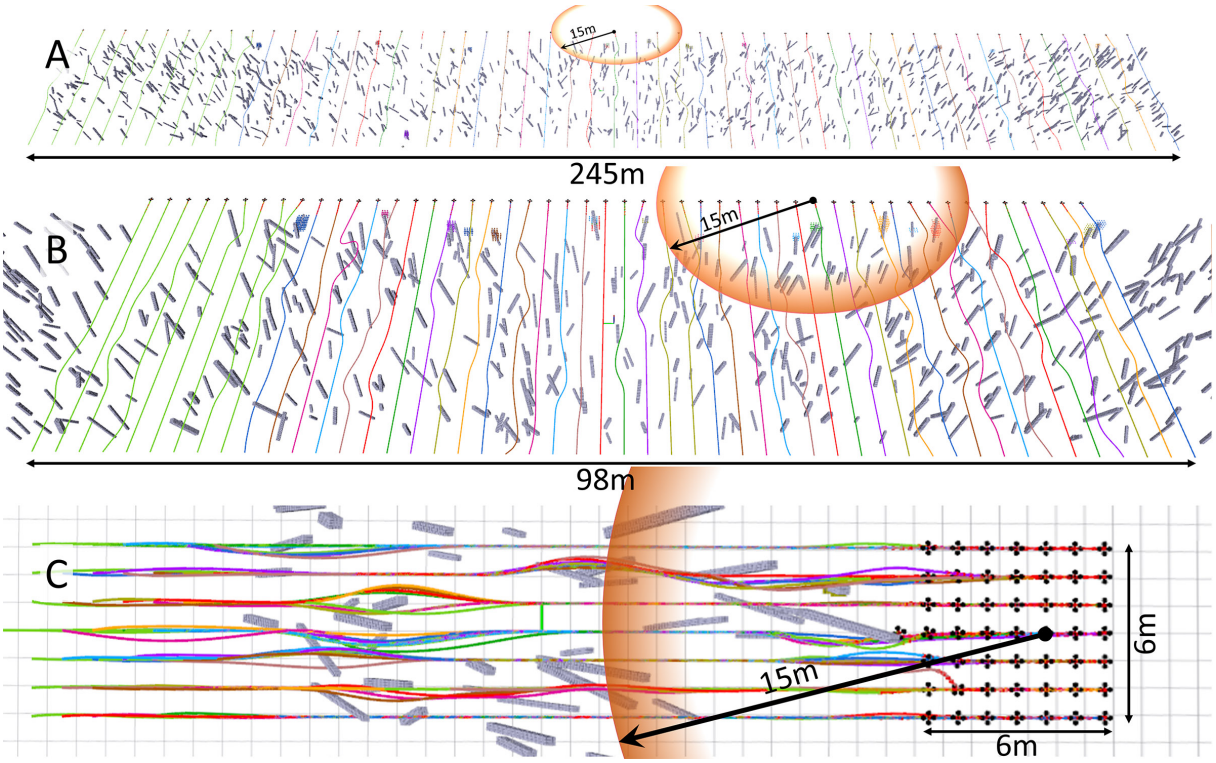

Fig. S15. Formation settings for scalability tests on the computation efficiency in Section *Time and Space Complexity*. (**A**) A loose formation. (**B**) A medium-intensity formation. (**C**) A tight formation. Orange circles indicate the range of two times the planning distance (7.5 m in Table S6).

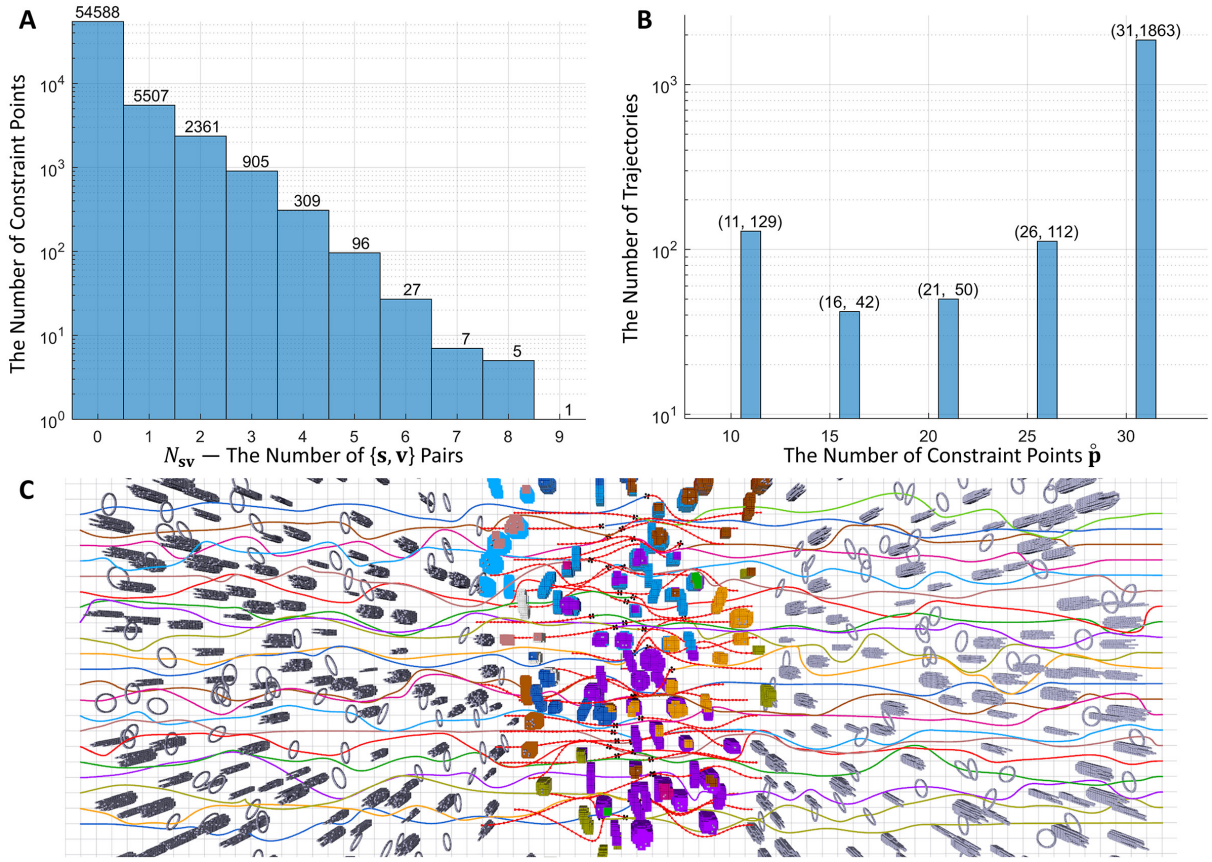

Fig. S16. Histograms of $N_{\mathbf{sv}}$ and the number of constraint points $\mathring{\mathbf{p}}$. (**A**) The statistic of $N_{\mathbf{sv}}$ on each constraint point. (**B**) The number of constraint points on each trajectory. (**C**) The test scenario.

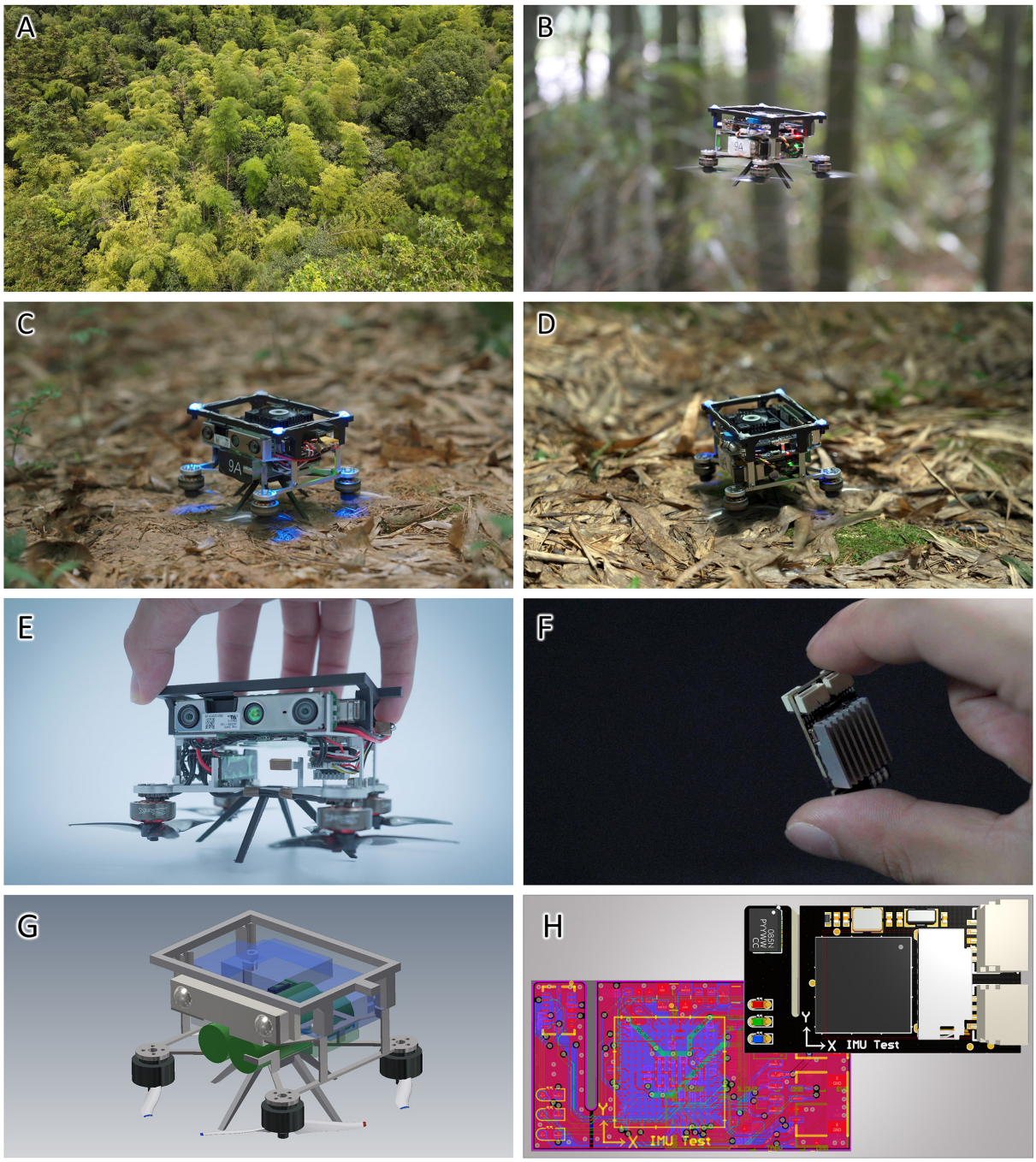

Fig. S17. More photographs of the environments and closeup views. (**A**) An aerial view of the bamboo forest where experiments are conducted. (**B-E**) Closeup views of the drone. (**F**) A miniature flight controller. (**G**) A mechanical model. (**H**) A PCB layout of the flight controller. Both (G) and (H) are released.

Tables

Table S6. Parameter settings used throughout this work. Capital letters A–H indicate various scenarios: **A** includes the 10-/40-drone swarm flight, circle position swap, circle random goals, goal select and set, moving obstacle avoidance, scalability test on drone number, and benchmark with various obstacle densities: **B** is the formation flight simulation, **C** is the tracking simulation, **D** is the velocity scalability benchmark, **E** is the real flight in the bamboo forest, **F** is the real-world formation flight, **G** is the real-world intensive reciprocal avoidance evaluation, and **H** is the real-world swarm tracking. Parameter λ_u is defined in Section *Penalty Formulation*. *Formation weight* λ_{fo} and *Tracking weight* λ_{tr} are not explicitly defined elsewhere, but they follow the same naming conventions as other weights.

Parameters	**Scenarios**	**Values**
Trajectory piece number M	All	5
Number of constraint points per piece κ	All	5
Smoothness weight λ_s	All	1.0
Total time weight λ_t	All	10.0
Obstacle avoidance weight λ_o	All	10000.0
Swarm collision avoidance weight λ_w	All	10000.0
Dynamical feasibility weight λ_d	All	10000.0
Uniform distribution weight λ_u	All	10000.0
Formation weight λ_{fo}	B, F	100.0
Tracking weight λ_{tr}	C, H	100.0
Obstacle clearance $\mathcal{C}_o$	A-D	0.3m
	E-H	0.2m
Swarm clearance $\mathcal{C}_w$	A-D	0.5m
	E-H	0.4m
Maximum velocity v_m	A-C	1.5m/s
	E-H	1.0m/s
	D	1/2/3m/s
Maximum acceleration a_m	All	6m/s^2
Maximum jerk j_m	All	10m/s^2
Planning distance	All	7.5m
Emergency stop time threshold	All	1s

Table S7. Component weights.

Component	Weight (g)
Onboard computer	69.1
RealSense D430	19.0
FCU	5.3
UWB	4.7
Battery	101.4
ESC	4.5
Four motors & propellers	42.8
WiFi adapter	3.4
Drone frame	19.5
Power management unit	6.5
Screws, cables, etc.	17.3
Total	**293.5**

Table S8. The values of sub-figure (1,1) in Fig. 7B.

Computing Time (MADER)			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
Median	0.10650	0.10860	0.14150
Lower Quartile	0.06390	0.06190	0.07790
Upper Quartile	0.15550	0.16600	0.20580
Computing Time (EGO-Swarm)			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
Median	0.00089	0.00120	0.00330
Lower Quartile	0.00050	0.00059	0.00079
Upper Quartile	0.00280	0.00370	0.01010
Computing Time (Proposed)			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
Median	0.00035	0.00041	0.00063
Lower Quartile	0.00017	0.00020	0.00034
Upper Quartile	0.00063	0.00068	0.00097

Table S9. The values of sub-figure (2,1) in Fig. 7B.

Computing Time (MADER)			
Velocity	**1.0**	**2.0**	**3.0**
Median	0.10650	0.10860	0.14150
Lower Quartile	0.06390	0.06190	0.07790
Upper Quartile	0.15550	0.16600	0.20580
Computing Time (EGO-Swarm)			
Velocity	**1.0**	**2.0**	**3.0**
Median	0.00081	0.00120	0.00210
Lower Quartile	0.00048	0.00059	0.00078
Upper Quartile	0.00240	0.00370	0.00620
Computing Time (Proposed)			
Velocity	**1.0**	**2.0**	**3.0**
Median	0.00043	0.00041	0.00029
Lower Quartile	0.00021	0.00020	0.00017
Upper Quartile	0.00078	0.00068	0.00065

Table S10. The values of sub-figures (1,2) and (2,2) in Fig. 7B.

Minimum Clearance			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
MADER	0.008	0.011	-0.184
EGO-Swarm	0.045	0.067	0.064
Proposed	0.086	0.065	0.058
Minimum Clearance			
Velocity	**1.0**	**2.0**	**3.0**
MADER	0.000	0.011	-0.086
EGO-Swarm	0.062	0.067	0.047
Proposed	0.068	0.065	0.064

Table S11. The values of sub-figures (1,3) and (2,3) in Fig. 7B.

Control Effort				
Obstacle Spacing		**2.0**	**1.5**	**1.0**
MADER	**Average**	6608.10	7575.70	8922.20
	SD	4254.50	4707.70	4958.20
EGO-Swarm	**Average**	624.30	523.94	1133.30
	SD	346.29	362.08	811.70
Proposed	**Average**	189.03	246.42	465.18
	SD	98.87	129.27	183.72
Control Effort				
Velocity		**1.0**	**2.0**	**3.0**
MADER	**Average**	743.52	7575.70	19994.00
	SD	1271.70	4707.70	7195.70
EGO-Swarm	**Average**	13.26	523.94	4176.30
	SD	10.29	362.08	2945.20
Proposed	**Average**	70.64	246.42	456.18
	SD	36.14	129.27	207.19

Table S12. The values of sub-figure (1,4) in Fig. 7B.

Flight Time (MADER)			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
Median	18.00	18.66	21.13
Lower Quartile	16.53	17.74	19.11
Upper Quartile	19.00	20.00	23.74
Flight Time (EGO-Swarm)			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
Median	23.12	23.40	24.15
Lower Quartile	21.47	22.14	22.82
Upper Quartile	24.69	24.82	26.35
Flight Time (Proposed)			
Obstacle Spacing	**2.0**	**1.5**	**1.0**
Median	21.25	20.79	20.75
Lower Quartile	20.00	20.14	19.70
Upper Quartile	22.27	22.58	22.71

Table S13. The values of sub-figure (2,4) in Fig. 7B.

Flight Time (MADER)			
Velocity	**1.0**	**2.0**	**3.0**
Median	35.00	18.66	14.65
Lower Quartile	33.03	17.73	13.41
Upper Quartile	36.09	20.00	16.15
Flight Time (EGO-Swarm)			
Velocity	**1.0**	**2.0**	**3.0**
Median	47.96	23.40	15.58
Lower Quartile	45.51	22.14	14.76
Upper Quartile	51.48	24.82	16.60
Flight Time (Proposed)			
Velocity	**1.0**	**2.0**	**3.0**
Median	35.37	20.79	17.72
Lower Quartile	34.19	20.14	16.66
Upper Quartile	37.45	22.58	18.93